

2025 下半年系统架构设计师案例分析回忆版-快速记忆

0. 来源

来源页面：

https://blog.csdn.net/cui_yonghua/article/details/154694201

页面标题：

2025年下半年系统架构设计师案例分析真题及答案解析回忆版

说明：

当前可抓取内容包含标题、目录和部分试题 1 内容；从试题 1 的问题 2 开始，页面提示需订阅专栏解锁全文。
本文档不复制付费正文，只基于可见目录和系统架构师案例分析常考方式，整理复习要点、答题模板和预测方向。

1. 页面目录对应考点

该页面目录显示的案例方向包括：

试题 1：质量属性
试题 2：Web 应用下单
试题 3：Redis 缓存
试题 4：嵌入式系统

对应复习重点：

试题方向	核心考点
质量属性	性能、可用性、安全性、可修改性、可靠性、可测试性
Web 应用下单	订单流程、库存一致性、支付幂等、事务、异步消息
Redis 缓存	缓存穿透、缓存击穿、缓存雪崩、缓存一致性、热点数据
嵌入式系统	实时性、可靠性、资源受限、软硬件协同、任务调度

2. 试题 1：质量属性

2.1 常见质量属性

性能：响应时间、吞吐量、并发数、资源利用率
可用性：故障恢复、冗余、容错、降级
可靠性：长期稳定运行、错误检测、故障隔离
安全性：认证、授权、加密、审计、防攻击
可修改性：低耦合、高内聚、模块化、接口稳定
可测试性：可观测、可注入、可模拟、日志和监控
可扩展性：横向扩展、分布式部署、容量扩展

2.2 质量属性场景模板

答题可以按 6 要素组织：

刺激源：谁触发
刺激：发生了什么
环境：在什么条件下
制品：影响哪个系统/模块
响应：系统如何处理
响应度量：用什么指标衡量

例子：

在促销高峰期间，大量用户同时提交订单，订单服务应在 3 秒内完成请求处理，并通过限流、缓存、异步队列和水平扩展保证系统吞吐量。

2.3 常见答题关键词

性能：

缓存、异步、负载均衡、连接池、数据库索引、读写分离、水平扩展

可用性：

集群、主备、故障转移、限流、降级、熔断、重试、超时控制

安全性：

身份认证、权限控制、数据加密、接口签名、防重放、日志审计

可修改性：

分层、模块化、接口隔离、配置化、插件化、微服务拆分

3. 试题 2：Web 应用下单

3.1 下单流程常见环节

用户提交订单
校验商品和价格
校验库存
创建订单
锁定或扣减库存
发起支付
支付回调
更新订单状态
发货或履约
通知用户

3.2 高频问题

库存一致性：

避免超卖
库存扣减要和订单状态配合
可使用库存预占、乐观锁、分布式锁、消息队列

支付幂等：

支付回调可能重复到达
必须用订单号或支付流水号做幂等控制
同一笔订单不能重复扣款或重复发货

事务边界：

订单、库存、支付跨系统时，通常不能依赖单机事务
可使用最终一致性、可靠消息、补偿机制、状态机

高并发：

- 前端防重复提交
- 后端限流
- 缓存商品信息
- 异步削峰
- 数据库索引优化
- 热点商品单独处理

3.3 答题模板

针对 Web 下单场景，我会将系统拆分为商品、订单、库存、支付和通知等模块。
下单时先校验商品、价格和用户状态，再创建待支付订单，并通过库存预占或乐观锁避免超卖。
支付回调采用幂等设计，按订单号和支付流水号去重。
对于高并发请求，通过缓存、限流、消息队列和异步处理削峰。
跨服务数据一致性采用最终一致性和补偿机制，避免长事务影响系统性能。

4. 试题 3：Redis 缓存

4.1 Redis 常见用途

- 热点数据缓存
- 会话缓存
- 排行榜
- 计数器
- 分布式锁
- 限流
- 消息队列辅助

4.2 三大缓存问题

缓存穿透：

查询不存在的数据，绕过缓存直接打到数据库。
解决：布隆过滤器、空值缓存、参数校验。

缓存击穿：

热点 Key 过期瞬间，大量请求同时访问数据库。
解决：互斥锁、热点 Key 永不过期、逻辑过期、提前刷新。

缓存雪崩：

大量 Key 同时过期或 Redis 故障，导致数据库压力骤增。
解决：过期时间加随机值、缓存集群、高可用、限流降级。

4.3 缓存一致性

常用策略：

- 先更新数据库，再删除缓存
- 延迟双删
- 消息队列异步删除
- 设置合理过期时间

注意：

不要轻易先更新缓存再更新数据库。
缓存通常不追求强一致，而是追求最终一致。

4.4 Redis 高可用

主从复制
哨兵 Sentinel
Cluster 集群
持久化 RDB/AOF
故障转移

4.5 答题模板

在高并发读场景中，可使用 Redis 缓存热点数据，降低数据库访问压力。
对不存在的数据使用布隆过滤器或空值缓存防止缓存穿透；
对热点 Key 使用互斥锁、逻辑过期或提前刷新防止缓存击穿；
对大量 Key 设置随机过期时间，并通过 Redis 集群和限流降级防止缓存雪崩。
缓存与数据库一致性可采用先更新数据库再删除缓存的策略，并结合消息队列和过期时间实现最终一致性。

5. 试题 4：嵌入式系统

5.1 嵌入式系统特点

专用性强
实时性要求高
资源受限
软硬件紧密结合
可靠性要求高
功耗敏感
环境适应性要求高

5.2 常见架构关注点

实时性：

任务调度、中断响应、优先级、截止时间

可靠性：

看门狗、冗余设计、故障检测、异常恢复

资源约束：

内存、CPU、存储、功耗、带宽

安全性：

固件升级、身份认证、通信加密、防篡改

可维护性：

模块化设计、远程升级、日志诊断、参数配置

5.3 答题模板

嵌入式系统设计应重点考虑实时性、可靠性和资源约束。
在架构上可采用分层设计，将硬件抽象层、驱动层、操作系统层、服务层和应用层分离。
对于实时任务，应根据任务优先级和截止时间进行调度，并保证关键中断的响应时间。
对于可靠性，可采用看门狗、故障检测、冗余和异常恢复机制。
对于资源受限问题，应优化内存占用、CPU 使用率和功耗，并支持远程升级和日志诊断。

6. 常见题型

6.1 填质量属性

题眼：

响应时间、吞吐量、并发量 -> 性能
故障恢复、不中断服务 -> 可用性
防止未授权访问 -> 安全性
修改功能影响小 -> 可修改性
容易定位问题、写测试 -> 可测试性

6.2 设计订单状态

常见状态：

待支付
已支付
已取消
已超时
已发货
已完成
退款中
已退款

6.3 解释缓存问题

答题顺序：

定义问题 -> 产生原因 -> 后果 -> 解决方案

6.4 嵌入式架构措施

答题顺序：

实时性 -> 可靠性 -> 资源优化 -> 安全升级 -> 监控诊断

7. 预测题

预测 1：质量属性场景

某系统要求在促销高峰时支持 5000 并发用户，下单响应时间不超过 2 秒。
该需求主要体现什么质量属性？应采取哪些架构措施？

参考答案：

主要体现性能，也涉及可扩展性。
措施：缓存、负载均衡、异步队列、数据库索引、读写分离、水平扩展、限流降级。

预测 2：支付回调幂等

支付平台可能多次回调同一订单，如何避免重复发货？

参考答案：

以订单号和支付流水号建立唯一约束或幂等表；
处理前检查订单状态；
只有待支付订单才能变更为已支付；
重复回调直接返回成功；
发货操作通过消息和状态机保证只执行一次。

预测 3：库存超卖

高并发秒杀场景下如何避免库存超卖？

参考答案：

可采用库存预扣、乐观锁、Redis 原子扣减、消息队列削峰、数据库唯一约束和最终一致性补偿。

预测 4：缓存雪崩

大量缓存同时失效导致数据库压力激增，是什么问题？如何解决？

参考答案：

缓存雪崩。

解决：过期时间加随机值、热点数据预热、Redis 高可用集群、限流、降级、熔断。

预测 5：嵌入式实时性

嵌入式控制系统要求关键任务必须在 10ms 内响应，应重点考虑什么架构设计？

参考答案：

重点考虑实时性。

措施：实时操作系统、优先级调度、中断优化、减少阻塞、关键任务独立、资源预分配。

8. 最终背诵版

质量属性题要先识别性能、可用性、安全性、可修改性等关键词，再给出架构措施。

Web 下单题重点关注订单流程、库存一致性、支付幂等、事务边界和高并发削峰。

Redis 缓存题重点掌握穿透、击穿、雪崩、一致性和高可用。

嵌入式系统题重点关注实时性、可靠性、资源受限、软硬件协同和安全升级。

答题时按“问题 -> 原因 -> 措施 -> 效果”组织，容易拿分。